

How to Use DynamoDB with Go

Learn how to use Amazon DynamoDB with Go using the AWS SDK v2, including table operations, CRUD with struct marshalling, queries, and batch operations.



By @nawazdhandala

Feb 12, 2026

6 min read

AWS

DynamoDB

Go

Golang

Go's strong typing and DynamoDB's schemaless nature might seem like an odd pairing, but the AWS SDK v2 for Go handles the translation beautifully. The `attributevalue` package marshals Go structs into DynamoDB items and back, similar to how `encoding/json` works. Let's build a complete data access layer.

Setting Up

Initialize your Go module and pull in the AWS SDK v2 packages.

Initialize the module

```
go mod init github.com/yourorg/dynamodb-api
```

Install the AWS SDK v2 modules

```
go get github.com/aws/aws-sdk-go-v2/config
go get github.com/aws/aws-sdk-go-v2/service/dynamodb
go get github.com/aws/aws-sdk-go-v2/feature/dynamodb/attributevalue
go get github.com/aws/aws-sdk-go-v2/feature/dynamodb/expression
go get github.com/google/uuid
```

Client Configuration

Create a shared DynamoDB client that you can pass around using dependency injection.

// db/client.go

```
package db
```

```
import (
```

```
"context"
"log"
"os"

"github.com/aws/aws-sdk-go-v2/config"
"github.com/aws/aws-sdk-go-v2/service/dynamodb"
)

// NewClient creates a configured DynamoDB client.
// It uses the default credential chain (env vars, shared config, IAM role).
func NewClient(ctx context.Context) *dynamodb.Client {
    cfg, err := config.LoadDefaultConfig(ctx,
        config.WithRegion(getRegion()),
    )
    if err != nil {
        log.Fatalf("unable to load AWS config: %v", err)
    }

    // Support endpoint override for local development
    endpoint := os.Getenv("DYNAMODB_ENDPOINT")
    if endpoint != "" {
        return dynamodb.NewFromConfig(cfg, func(o *dynamodb.Options) {
            o.BaseEndpoint = &endpoint
        })
    }

    return dynamodb.NewFromConfig(cfg)
}

func getRegion() string {
    if r := os.Getenv("AWS_REGION"); r != "" {
        return r
    }
    return "us-east-1"
}
```

Defining Your Model

Define a Go struct with `dynamodbav` tags for field mapping. These tags work like `json` tags but for DynamoDB attributes.

```
// models/user.go
package models

import "time"

// User represents a user record in DynamoDB.
// The dynamodbav tags control how struct fields map to DynamoDB attributes.
type User struct {
    UserID    string    `dynamodbav:"user_id"`
    Name      string    `dynamodbav:"name"`
    Email     string    `dynamodbav:"email"`
    Status    string    `dynamodbav:"status"`
    Age       int       `dynamodbav:"age,omitempty"`
    Tags      []string  `dynamodbav:"tags,omitempty"`
    CreatedAt time.Time `dynamodbav:"created_at"`
    UpdatedAt time.Time `dynamodbav:"updated_at"`
}

// CreateUserInput holds the fields needed to create a new user.
type CreateUserInput struct {
    Name string
```

```
Email string
Age    int
Tags   []string
}
```

The Repository

Build a repository that handles all CRUD operations.

```
// repository/user_repo.go
package repository

import (
    "context"
    "fmt"
    "time"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/feature/dynamodb/attributevalue"
    "github.com/aws/aws-sdk-go-v2/feature/dynamodb/expression"
    "github.com/aws/aws-sdk-go-v2/service/dynamodb"
    "github.com/aws/aws-sdk-go-v2/service/dynamodb/types"
    "github.com/google/uuid"

    "github.com/yourorg/dynamodb-api/models"
)

const tableName = "Users"

// UserRepository handles DynamoDB operations for users.
type UserRepository struct {
    client *dynamodb.Client
}

// NewUserRepository creates a new repository instance.
func NewUserRepository(client *dynamodb.Client) *UserRepository {
    return &UserRepository{client: client}
}

// Create inserts a new user into DynamoDB.
func (r *UserRepository) Create(ctx context.Context, input models.CreateUserInput) (*models.User, error) {
    now := time.Now().UTC()
    user := models.User{
        UserID:    uuid.New().String(),
        Name:      input.Name,
        Email:     input.Email,
        Status:    "active",
        Age:      input.Age,
        Tags:     input.Tags,
        CreatedAt: now,
        UpdatedAt: now,
    }

    // Marshal the Go struct into a DynamoDB attribute map
    item, err := attributevalue.MarshalMap(user)
    if err != nil {
        return nil, fmt.Errorf("failed to marshal user: %w", err)
    }

    _, err = r.client.PutItem(ctx, &dynamodb.PutItemInput{
        TableName: aws.String(tableName),
```

```

        Item:      item,
        // Prevent overwriting if the ID somehow already exists
        ConditionExpression: aws.String("attribute_not_exists(user_id)",
    })
    if err != nil {
        return nil, fmt.Errorf("failed to put item: %w", err)
    }

    return &user, nil
}

// GetByID retrieves a user by their partition key.
func (r *UserRepository) GetByID(ctx context.Context, userID string) (*models.User, error) {
    result, err := r.client.GetItem(ctx, &dynamodb.GetItemInput{
        TableName: aws.String(tableName),
        Key: map[string]types.AttributeValue{
            "user_id": &types.AttributeValueMemberS{Value: userID},
        },
        ConsistentRead: aws.Bool(true),
    })
    if err != nil {
        return nil, fmt.Errorf("failed to get item: %w", err)
    }

    // Item not found
    if result.Item == nil {
        return nil, nil
    }

    // Unmarshal the DynamoDB item back into a Go struct
    var user models.User
    err = attributevalue.UnmarshalMap(result.Item, &user)
    if err != nil {
        return nil, fmt.Errorf("failed to unmarshal user: %w", err)
    }

    return &user, nil
}

// Update modifies specific fields on an existing user.
func (r *UserRepository) Update(ctx context.Context, userID string, name string, email string)
(*models.User, error) {
    // Build the update expression using the expression builder
    // This is safer than building strings manually
    update := expression.Set(
        expression.Name("name"), expression.Value(name),
    ).Set(
        expression.Name("email"), expression.Value(email),
    ).Set(
        expression.Name("updated_at"), expression.Value(time.Now().UTC()),
    )

    // Add a condition to ensure the item exists
    condition := expression.AttributeExists(expression.Name("user_id"))

    expr, err := expression.NewBuilder().
        WithUpdate(update).
        WithCondition(condition).
        Build()
    if err != nil {
        return nil, fmt.Errorf("failed to build expression: %w", err)
    }

    result, err := r.client.UpdateItem(ctx, &dynamodb.UpdateItemInput{
        TableName: aws.String(tableName),
        Key: map[string]types.AttributeValue{
            "user_id": &types.AttributeValueMemberS{Value: userID},

```

```

    },
    UpdateExpression:      expr.Update(),
    ExpressionAttributeNames: expr.Names(),
    ExpressionAttributeValues: expr.Values(),
    ConditionExpression:    expr.Condition(),
    ReturnValues:          types.ReturnValueAllNew,
  })
  if err != nil {
    return nil, fmt.Errorf("failed to update item: %w", err)
  }

  var user models.User
  err = attributevalue.UnmarshalMap(result.Attributes, &user)
  if err != nil {
    return nil, fmt.Errorf("failed to unmarshal updated user: %w", err)
  }

  return &user, nil
}

// Delete removes a user and returns the deleted item.
func (r *UserRepository) Delete(ctx context.Context, userID string) (*models.User, error) {
  result, err := r.client.DeleteItem(ctx, &dynamodb.DeleteItemInput{
    TableName: aws.String(tableName),
    Key: map[string]types.AttributeValue{
      "user_id": &types.AttributeValueMemberS{Value: userID},
    },
    ReturnValues: types.ReturnValueAllOld,
  })
  if err != nil {
    return nil, fmt.Errorf("failed to delete item: %w", err)
  }

  if result.Attributes == nil {
    return nil, nil
  }

  var user models.User
  err = attributevalue.UnmarshalMap(result.Attributes, &user)
  if err != nil {
    return nil, fmt.Errorf("failed to unmarshal: %w", err)
  }

  return &user, nil
}

// QueryByStatus finds users by status using a GSI.
func (r *UserRepository) QueryByStatus(ctx context.Context, status string, limit int32) (
  []models.User, error) {
  keyCond := expression.KeyEqual(
    expression.Key("status"), expression.Value(status),
  )

  expr, err := expression.NewBuilder().
    WithKeyCondition(keyCond).
    Build()
  if err != nil {
    return nil, fmt.Errorf("failed to build expression: %w", err)
  }

  result, err := r.client.Query(ctx, &dynamodb.QueryInput{
    TableName:      aws.String(tableName),
    IndexName:      aws.String("status-index"),
    KeyConditionExpression: expr.KeyCondition(),
    ExpressionAttributeNames: expr.Names(),
    ExpressionAttributeValues: expr.Values(),
    Limit:          &limit,
  })

```

```
    })
    if err != nil {
        return nil, fmt.Errorf("failed to query: %w", err)
    }

    var users []models.User
    err = attributevalue.UnmarshalListOfMaps(result.Items, &users)
    if err != nil {
        return nil, fmt.Errorf("failed to unmarshal results: %w", err)
    }

    return users, nil
}
```

Batch Operations

For bulk operations, use `BatchWriteItem` to process up to 25 items at a time, and retry any unprocessed items that DynamoDB returns.

```
// BatchCreate writes multiple users in a single batch operation.
func (r *UserRepository) BatchCreate(ctx context.Context, users []models.User) error {
    // DynamoDB limits batch writes to 25 items per request
    const batchSize = 25

    for i := 0; i < len(users); i += batchSize {
        end := i + batchSize
        if end > len(users) {
            end = len(users)
        }

        batch := users[i:end]
        writeRequests := make([]types.WriteRequest, len(batch))

        for j, user := range batch {
            item, err := attributevalue.MarshalMap(user)
            if err != nil {
                return fmt.Errorf("failed to marshal user %s: %w", user.UserID, err)
            }
            writeRequests[j] = types.WriteRequest{
                PutRequest: &types.PutRequest{Item: item},
            }
        }

        requestItems := map[string][]types.WriteRequest{
            tableName: writeRequests,
        }
        backoff := 100 * time.Millisecond
        const maxRetries = 5

        for attempt := 0; ; attempt++ {
            result, err := r.client.BatchWriteItem(ctx, &dynamodb.BatchWriteItemInput{
                RequestItems: requestItems,
            })
            if err != nil {
                return fmt.Errorf("batch write failed: %w", err)
            }

            if len(result.UnprocessedItems) == 0 {
                break
            }
            if attempt == maxRetries {
                return fmt.Errorf("batch write still had unprocessed items after %d retries", maxRetries)
            }
        }
    }
}
```

```
        }

        time.Sleep(backoff)
        backoff *= 2
        requestItems = result.UnprocessedItems
    }
}

return nil
}
```

Putting It Together

Here's a main function that demonstrates the repository in action.

```
// main.go
package main

import (
    "context"
    "fmt"
    "log"

    "github.com/yourorg/dynamodb-api/db"
    "github.com/yourorg/dynamodb-api/models"
    "github.com/yourorg/dynamodb-api/repository"
)

func main() {
    ctx := context.Background()

    // Initialize the DynamoDB client
    client := db.NewClient(ctx)
    userRepo := repository.NewUserRepository(client)

    // Create a user
    user, err := userRepo.Create(ctx, models.CreateUserInput{
        Name:   "Alice Johnson",
        Email:  "alice@example.com",
        Tags:   []string{"admin", "beta-tester"},
    })
    if err != nil {
        log.Fatalf("create failed: %v", err)
    }
    fmt.Printf("Created user: %s\n", user.UserID)

    // Read the user back
    found, err := userRepo.GetByID(ctx, user.UserID)
    if err != nil {
        log.Fatalf("get failed: %v", err)
    }
    fmt.Printf("Found user: %s (%s)\n", found.Name, found.Email)

    // Query by status
    activeUsers, err := userRepo.QueryByStatus(ctx, "active", 10)
    if err != nil {
        log.Fatalf("query failed: %v", err)
    }
    fmt.Printf("Found %d active users\n", len(activeUsers))
}
```

The AWS SDK v2 for Go is well-designed and idiomatic. The expression builder prevents injection-style bugs in your DynamoDB expressions, and the `attributevalue` package makes struct marshalling painless. Use the repository pattern to keep your DynamoDB logic testable and isolated from your HTTP handlers.

For monitoring your DynamoDB tables from Go applications, check out our guide on [monitoring DynamoDB with CloudWatch alarms](#).

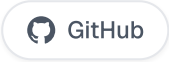
Share this article



Nawaz Dhandala Author

@nawazdhandala • Feb 12, 2026 • 6 min read

Nawaz is building OneUptime with a passion for engineering reliable systems and improving observability.



Technically validated · Jun 01, 2026

[View report >](#)



Help improve this post

Every OneUptime blog post is open source. Found a typo, an inaccuracy, or have a clearer way to explain something? Anyone can contribute — your edits make this post better for everyone who reads it next.

[Edit this post on GitHub](#)

[Contributing guidelines](#)

[Open source >](#)

OneUptime is the Open-Source Observability Platform

Your complete reliability stack unified: infrastructure monitoring, incident management, status pages, and APM. Open-source and self-hostable.

[Get started for free](#)

[Request a demo](#)



Status Page
Real-time status updates



Incidents
Detect and resolve fast



Monitoring
Monitor any resource



On-Call
Smart alert routing



Maintenance
Plan & communicate downtime



Logs
Fastest log ingest and search



Metrics
Performance insights



Traces
End-to-end distributed tracing



Exceptions
Catch and fix bugs early



Workflows
Automate any process



Dashboards
Visualize all your data



Kubernetes
Monitor K8s clusters



Profiles
CPU & memory profiling



AI
Detect, diagnose, and resolve incidents with AI-powered root cause analysis and code fixes.

>

<https://oneuptime.com/blog/post/2026-02-12-use-dynamodb-with-go/view>

9/9